02285 AI and MAS, F26
Exercises for week 3, 17/2-26
SOLUTIONS

Exercise 1 (Planes and airports)

Given the action schemas from Figure 1, what are all the applicable concrete instances of $Fly(p, from, to)$ in the state described by

$$At(P1, JFK) \land At(P2, SFO) \land Plane(P1) \land Plane(P2) \land$$
$$Airport(JFK) \land Airport(SFO)?$$

SOLUTION.

$$Fly(P_1, JFK, SFO), Fly(P_1, JFK, JFK),$$
$$Fly(P_2, SFO, JFK), Fly(P_2, SFO, SFO).$$

Exercise 2 (Simplified Gripper domain)

This exercise refers to the simplified Gripper domain introduced in the lecture on Classical Automated Planning, cf. the slides.

a) The domain was in the lecture declared using two symmetric actions $MoveAB(x)$ and $MoveBA(x)$. Write the actions schema for a single action *Move* that is able to replace $MoveAB$ and $MoveBA$. You might have to introduce additional variables and predicates.

SOLUTION.

ACTION : $Move(x, y, z)$
PRECONDITION : $Box(x) \land Location(y) \land Location(z) \land y \neq z \land In(x, y)$
EFFECT : $In(x, z) \land \neg In(x, y)$

b) Using *Move* instead of $MoveAB$ and $MoveBA$ is likely to affect also the description of the initial state. Provide the new initial state of the simplified Gripper planning problem with n boxes initially located in location A.

$Init(At(C_1,\ SFO) \ \wedge\ At(C_2,\ JFK) \ \wedge\ At(P_1,\ SFO) \ \wedge\ At(P_2,\ JFK)$
$\quad \wedge\ Cargo(C_1) \ \wedge\ Cargo(C_2) \ \wedge\ Plane(P_1) \ \wedge\ Plane(P_2)$
$\quad \wedge\ Airport(JFK) \ \wedge\ Airport(SFO))$
$Goal(At(C_1,\ JFK) \ \wedge\ At(C_2,\ SFO))$
$Action(Load(c,\ p,\ a),$
$\quad$ PRECOND: $At(c,\ a) \ \wedge\ At(p,\ a) \ \wedge\ Cargo(c) \ \wedge\ Plane(p) \ \wedge\ Airport(a)$
$\quad$ EFFECT: $\neg\ At(c,\ a) \ \wedge\ In(c,\ p))$
$Action(Unload(c,\ p,\ a),$
$\quad$ PRECOND: $In(c,\ p) \ \wedge\ At(p,\ a) \ \wedge\ Cargo(c) \ \wedge\ Plane(p) \ \wedge\ Airport(a)$
$\quad$ EFFECT: $At(c,\ a) \ \wedge\ \neg\ In(c,\ p))$
$Action(Fly(p,\ from,\ to),$
$\quad$ PRECOND: $At(p,\ from) \ \wedge\ Plane(p) \ \wedge\ Airport(from) \ \wedge\ Airport(to)$
$\quad$ EFFECT: $\neg\ At(p,\ from) \ \wedge\ At(p,\ to))$

Figure 1: A PDDL description of an air cargo transportation planning problem.

SOLUTION. Initial state:

$$Location(A) \wedge Location(B) \wedge \bigwedge_{i=1,\ldots,n} Box(i) \wedge \bigwedge_{i=1,\ldots,n} In(i, A)$$

c) Assume s is a state resulting from the execution of some sequence of *Move* actions in the initial state. Show that if $In(\alpha, \beta)$ is a literal in s, then α is a constant denoting a box and β is a constant denoting a location. (If this doesn't hold, go back to the first question and revise your action schema).

SOLUTION. Let $P(s)$ denote the following property: If $In(\alpha, \beta)$ is a literal in s, then α is a constant denoting a box and β is a constant denoting a location. In our formalisation, the constants denoting boxes are $1, \ldots, n$ and the constants denoting locations are A and B. Hence $P(s)$ is equivalent to the following property: If $In(\alpha, \beta)$ is a literal in s, then $\alpha \in \{1, \ldots, n\}$ and $\beta \in \{A, B\}$. Let s_0 denote the initial state.

We need to prove the following: 1) $P(s_0)$ is true; 2) If s' is the result of a non-empty sequence of a *Move* actions applied to s_0, then $P(s')$ is true. It is clear that $P(s_0)$ is true, consulting the answer to the previous question. To prove 2, we use the action schema of *Move* provided in question a. If $In(\alpha, \beta)$ is added by a (ground) $Move(\alpha, \gamma, \beta)$ action applied in a state s, then $Box(\alpha)$, $Location(\gamma)$ and $Location(\beta)$ must all hold in s (since they are precondition literals). Note that *Move* does not affect the *Box* and *Location* literals, so if $Box(\alpha)$, $Location(\gamma)$ and $Location(\beta)$ hold in s, then they also hold in s_0 (we assume s is achieved by a sequence of *Move* actions applied to s_0). By construction of s_0 this implies $\alpha \in \{1, \ldots, n\}$ and $\beta, \gamma \in \{A, B\}$. This proves 2.

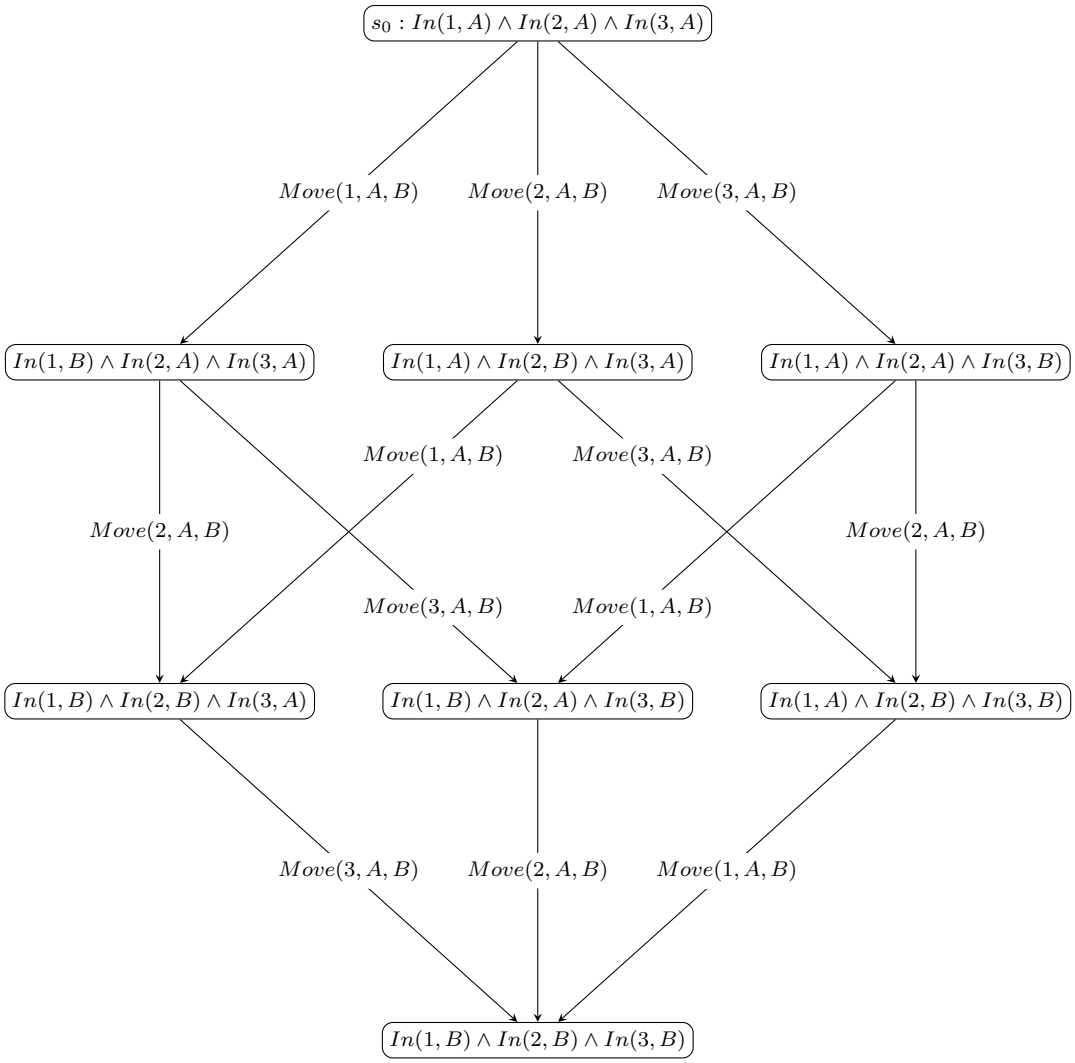
Note: If the action schema of question a didn't include the *Location* preconditions, the property would not hold. Then we could execute $Move(1, A, 1)$ and get $In(1, 1)$.

d) Does you *Move* action allow moving a box from a location to itself, e.g. moving box 1 from location A to location A (in one action)? If so, revise your action schema.

SOLUTION. A box can not be moved from a location to itself, since we have required $y \neq z$ in the precondition.

e) Draw a diagram of the state space of the simplified Gripper planning problem with $n = 3$. To save space, you can exclude mentioning the rigid predicates in your states. A **rigid predicate** is one that is state invariant like Box in the simplified Griper problem used in the slides. More precisely, an n-ary predicate P is *rigid* if $P(c_1, \ldots, c_n)$ for any choice of constants $c_1, \ldots, c_n$ will always have the same truth value as in the initial state, independent of the actions applied.

SOLUTION. We omit the rigid predicates Box and $Location$. Also we omit the edges point upwards: If there is a downwards edge $Move(i, A, B)$, then there is a symmetrical upwards edge $Move(i, B, A)$.



f) The *branching factor* of a planning problem is the maximum number of actions applicable in any state reachable from the initial state. What is the branching factor of the simplified Gripper problem as a function of n?

SOLUTION. The branching factor is n. In fact, in every state there is *exactly* n applicable actions. In the state space shown above, we have $n = 3$, and each state has 3 outgoing edges (when we remember to also count the omitted upward arrows).

g) Consider a modified planning problem with three locations, A, B and C. It is assumed that any box can be moved from any location to any other location. Describe the planning problem where all n boxes initially are located in A and where the goal is to get the even-numbered boxes into B and the odd-numbered into C. How much do you need to change: Initial state? Action schema? Goal state?

SOLUTION. To add a new location C, it is required to add $Location(C)$ as a conjunct of the initial state. Otherwise, the initial state is as before. The action schema is unchanged. The goal becomes:

$$\bigwedge\{In(i, B) \mid i \in \{1, \ldots, n\} \text{ is even}\} \wedge \bigwedge\{In(j, C) \mid j \in \{1, \ldots, n\} \text{ is odd}\}.$$

h) Modify the planning problem from the previous question further so that a single *Move* can only move a box from A to B, from B to A, from B to C or from C to B. You might again need additional predicates.

SOLUTION. We add a new binary predicate *Connected*. Then we add the following conjuncts to the initial state:

$$Connected(A, B) \wedge Connected(B, A) \wedge Connected(B, C) \wedge Connected(C, B).$$

We modify the action schema *Move* from question a) by adding the following precondition literal: $Connected(y, z)$. No other changes are necessary, in particular we don't need to change the goal description.

Exercise 3 (Blocks World)

Consider the Blocks World problem P described in PDDL in Figure 2. Let s_0 denote the initial state of this problem, and let g denote the goal.

a) The blocks world P problem possesses the so-called **Sussman anomaly**. The problem was considered anomalous because the non-interleaved planners of the early 1970s could not solve it. A non-interleaved planner is a planner that, given two subgoals g_1 and g_2, produces either a plan for g_1 concatenated with a plan for g_2, or vice versa. Explain why a non-interleaved planner cannot solve this problem.

SOLUTION. A non-interleaved planner will either first try to satisfy $On(A, B)$ or $On(B, C)$. If it goes for $On(A, B)$ first, C will be moved to the table and A on top of B. Then to reach the second goal, $On(B, C)$, A will simply be moved to the table and B on top of C. This is not a solution. A similar argument goes if $On(B, C)$ is satisfied first.

b) Determine $h^*(s_0)$ (the optimal cost of a solution to the problem).

$Init(On(A, Table) \wedge On(B, Table) \wedge On(C, A)$
$\quad \wedge Block(A) \wedge Block(B) \wedge Block(C) \wedge Clear(B) \wedge Clear(C))$
$Goal(On(A, B) \wedge On(B, C))$
$Action(Move(b, x, y),$
PRECOND: $On(b, x) \wedge Clear(b) \wedge Clear(y) \wedge Block(b) \wedge Block(y) \wedge$
$(b \neq x) \wedge (b \neq y) \wedge (x \neq y),$
EFFECT: $On(b, y) \wedge Clear(x) \wedge \neg On(b, x) \wedge \neg Clear(y))$
$Action(MoveToTable(b, x),$
PRECOND: $On(b, x) \wedge Clear(b) \wedge Block(b) \wedge (b \neq x),$
EFFECT: $On(b, Table) \wedge Clear(x) \wedge \neg On(b, x))$

Figure 10.3 A planning problem in the blocks world: building a three-block tower. One solution is the sequence $[MoveToTable(C, A), Move(B, Table, C), Move(A, Table, B)]$.

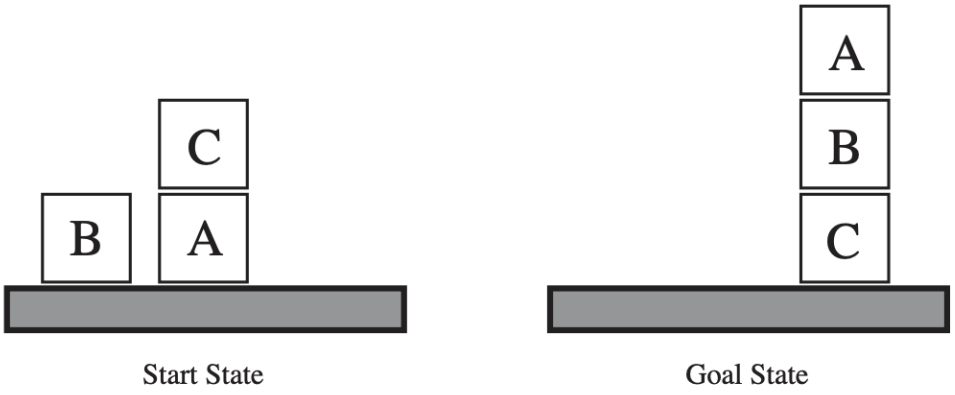


Figure 10.4 Diagram of the blocks-world problem in Figure 10.3.

Figure 2: A simple blocks world problem from R&N, 3ed.

SOLUTION. An optimal solution is $MoveToTable(C, A), Move(B, Table, C), Move(A, Table, B)$. So $h^*(s_0) = 3$.

c) Determine $h_{gc}(s_0)$ (the goal count heuristic).

SOLUTION. The goal is $On(A, B) \wedge On(B, C)$. None of them is initially true, so $h_{gc}(s_0) = 2$.

d) Provide an example of a Blocks World problem where $h_{gc}(s_0) = 1$ and $h^*(s_0) \geq 1000$ (that is, a problem where h_{gc} heavily underestimates the optimal cost).

SOLUTION. Initial state: 1000 blocks on top of each other where only the bottom one is misplaced. Then $h_{gc} = 1$. But all 999 blocks on top of it has to be moved away first to place it correctly.

e) Provide an example of a Blocks World problem where $h_{gc}(s_0) > h^*(s_0)$ (that is, where h_{gc} **over**estimates the optimal cost).

SOLUTION. Take the initial state from Figure 10.3. Let the goal be $On(C, Table) \wedge \neg On(C, A)$. Then the initial state has two unsatisfied goal literals, so $h_{gc}(s_0) = 2$, but it only takes one action to satisfy both, $h^*(s_0) = 1$.

f) Determine $h_{ip}(s_0)$ (the ignore preconditions and non-goal literal heuristic) for the original Blocks World problem of Figure 10.3.

SOLUTION. By ignoring preconditions, we can move A on top of B in one action, and B on top of C in one action. Hence $h_{ip}(s_0) = 2$.

g) Provide an example of a Blocks World problem where $h_{ip}(s_0) = 1$ and $h^*(s_0) \geq 1000$ (that is, a problem where h_{ip} heavily underestimates the optimal cost).

SOLUTION. Same example as for h_{gc} will work: 1000 blocks with only one misplaced at the bottom. According to the relaxation, the misplaced block can be placed correctly in one action (since preconditions are ignored).

h) Is it possible to create a Blocks World problem where $h_{ip}(s_0) > h^*(s_0)$?

SOLUTION. No, the heuristic is admissible.

i) In the initial state of the Blocks World problem of Figure 10.3, the ground atoms of the form $C_1 = C_2$ that hold true are left implicit. Add the relevant atoms of this form to the initial state. In the rest of the exercise, we will assume that these atoms have been added.

SOLUTION. There are only four constants A, B, C and $Table$, so the relevant atoms are: $A = A$, $B = B$, $C = C$, $Table = Table$.

j) The heuristic based on delete-relaxation requires that we have a planning problem where goals and preconditions only contain positive literals. This is not true of the Blocks World problem of Figure 10.3, as an inequality $x \neq y$ is short for $\neg(x = y)$. Rephrase the Blocks World problem so that there are no negative literals in goal and preconditions. In the rest of the exercise, we will work with these rephrased version of the problem.

SOLUTION. We treat $\neq$ as a new predicate (not as short for a negated equality). But then we also have to add more atoms to the initial state: $C_1 \neq C_2$ for each distinct pair constants $C_1, C_2 \in A, B, C, Table$.

k) Determine $h^+(s_0)$ (the delete-relaxation heuristic) for the rephrased Blocks World problem.

SOLUTION. We cannot initially move A onto B, as A is not clear. So we need to move C to the table first. Then we still need two actions to reach the goal: B on top of C and A on top of B. So $h^+(s_0) = 3$.

l) Provide an example of a Blocks World problem where $h^+(s_0) < h^*(s_0)$. Explain what it is that make h^+ underestimate the optimal cost in your example.

SOLUTION. Let the initial state be $On(C, A) \wedge On(A, B)$ and the goal be as in the original problem. Then a solution in the delete-relaxed problem is:

$$MoveToTable(C, A), MoveToTable(A, B), Move(B, Table, C).$$

The point is this: A is initially on top of B, so if we ignore delete-lists, we simply have to ensure that B gets on top of C. And that can be done in the three steps given. In the original, non-relaxed problem, we however face the challenge that when achieving the subgoal $On(B, C)$ we destroy the other subgoal $On(A, B)$ that held initially (it occurs as a negative effect in the second action above). Hence we need an extra action in the non-relaxed problem.

m) As defined earlier, a *rigid predicate* is one that is state variant, that is, a predicate P such that no $P(c_1, \ldots, c_n)$ can ever get a different truth value than the one it has in the initial state. Find the rigid predicates in the rephrased Blocks World problem. Then show that for any atom $P(c_1, \ldots, c_n)$ in any planning problem where P is rigid, we must have either $h_{add}(P(c_1, \ldots, c_n), s) = 0$ or $h_{add}(P(c_1, \ldots, c_n), s) = \infty$ for all states s, where h_{add} is the additive heuristic.

SOLUTION. The rigid predicates are everything of the form equality ($=$), inequality ($\neq$) and $Block$. The remaining predicates, $Clear$ and On are not rigid. If P is rigid and $P(c_1, \ldots, c_n)$ is true in the initial state s_0 of a planning problem, then it is necessarily true in any state, by definition of rigidity. Since $P(c_1, \ldots, c_n)$ is true in s_0, we get $h_{add}(P(c_1, \ldots, c_n), s_0) = 0$ by definition of h_{add}. It then follows that $h_{add}(P(c_1, \ldots, c_n), s) = 0$ for any state s. Now assume $P(c_1, \ldots, c_n)$ is false in the initial state. Since P is rigid, $P(c_1, \ldots, c_n) \notin \textsc{Add}(a)$ for every actions a. From the definition of h_{add} we now get $h_{add}(P(c_1, \ldots, c_n), s_0) = \infty$, and hence, by rigidity, $h_{add}(P(c_1, \ldots, c_n), s) = \infty$ for all states s.

n) Determine $h_{add}(s_0)$ (the additive heuristic) for the rephrased Blocks World problem. Explain also the intuitive meaning of all the values $h_{add}(p, s_0)$ you calculate in order to compute $h_{add}(s_0)$. *Hint*: Make use of your answer to the previous question. You can simplify the calculations by not including all the calculations of the atoms using rigid predicates, but simply immediately replace by 0 or ∞ whenever appropriate.

SOLUTION.

$$\begin{aligned} h_{add}(s_0) = h_{add}(g, s_0) &= h_{add}(On(A,B) \wedge On(B,C), s_0) \\ &= h_{add}(On(A,B), s_0) + h_{add}(On(B,C), s_0). \end{aligned}$$

We need to calculate $h_{add}(On(A,B), s_0)$ and $h_{add}(On(B,C), s_0)$.

$$\begin{aligned} &h_{add}(On(A,B), s_0) = \\ &1 + \min\{h_{add}(On(A,x) \wedge Clear(A) \wedge Clear(B) \wedge Block(A) \wedge \\ &\quad Block(B) \wedge A \neq x \wedge A \neq B \wedge x \neq B, s_0) \mid x \text{ is a constant}\} \\ &1 + \min\{h_{add}(On(A,x), s_0) + h_{add}(Clear(A), s_0) + h_{add}(Clear(B), s_0) + \\ &\quad h_{add}(Block(A), s_0) + h_{add}(Block(B), s_0) + h_{add}(A \neq x, s_0) \\ &\quad + h_{add}(A \neq B, s_0) + h_{add}(x \neq B, s_0) \mid x \text{ is a constant } \} = \\ &1 + \min\{h_{add}(On(A,C), s_0) + h_{add}(Clear(A), s_0), \\ &\quad h_{add}(On(A,Table), s_0) + h_{add}(Clear(A), s_0)\} = \\ &1 + \min\{h_{add}(On(A,C), s_0) + h_{add}(Clear(A), s_0), \\ &\quad 0 + h_{add}(Clear(A), s_0)\} = \\ &1 + h_{add}(Clear(A), s_0) = \\ &1 + (1 + h_{add}(On(C,A), s_0) + h_{add}(Clear(C), s_0)) = 1 + (1 + 0) = 2. \end{aligned}$$

The intuitive meaning of $h_{add}(On(A,B), s_0) = 2$ is that A can get on top of B in two steps: Move C to the table (to make A clear) then move A on top of A. Calculating $h_{add}(On(B,C), s_0)$ is similar:

$$\begin{aligned} &h_{add}(On(B,C), s_0) = \\ &1 + \min\{h_{add}(On(B,x) \wedge Clear(B) \wedge Clear(C) \wedge Block(A) \wedge \\ &\quad Block(B) \wedge B \neq x \wedge B \neq C \wedge x \neq C, s_0) \mid x \text{ is a constant}\} \\ &1 + \min\{h_{add}(On(B,x), s_0) + h_{add}(Clear(B), s_0) + \\ &\quad h_{add}(Clear(C), s_0) \mid x \text{ is a constant} \neq B, C\} = \\ &1 + \min\{h_{add}(On(B,A), s_0), h_{add}(On(B,Table), s_0)\} \\ &1 + 0 = 1. \end{aligned}$$

The intuitive meaning of $h_{add}(On(B,C), s_0) = 1$ is that we can get B on top of C in one move: $Move(B, Table, C)$. Now we can finally calculate $h_{add}(s_0) = h_{add}(On(A,B), s_0) + h_{add}(On(B,C), s_0) = 2 + 1 = 3$.

o) Give an example of a Blocks World problem where $h_{add}(s_0) > h^*(s_0)$.

SOLUTION. Take the initial state to be $On(A,B) \wedge On(B,C)$. Take the goal to be $On(A,Table) \wedge On(B,Table)$. Then $h_{add}(On(A,Table), s_0) = 1$ and $h_{add}(On(B,Table), s_0) = 2$ (no action with effect $On(B,Table)$ has all its preconditions satisfied in s_0). Hence we get $h_{add}(g, s_0) = 1 + 2 = 3$. However, $h^*(s_0) = 2$: Move A to table, then B to table.

p) Determine $h_{max}(s_0)$ (the max heuristic) for the rephrased Blocks World problem. Explain also the intuitive meaning of all the values $h_{max}(p, s_0)$ you calculate in order to compute $h_{max}(s_0)$.

SOLUTION.

$$\begin{aligned} h_{max}(s_0) = h_{max}(g, s_0) &= h_{max}(On(A,B) \wedge On(B,C), s_0) \\ &= \max\{h_{max}(On(A,B), s_0), h_{max}(On(B,C), s_0)\}. \end{aligned}$$

It is easy to see that $h_{max}(On(B,C),s_0) = 1$. Let's calculate $h_{max}(On(A,B),s_0)$:

$$
\begin{aligned}
&h_{max}(On(A,B),s_0) = \\
&1 + \min\{h_{max}(On(A,x) \wedge Clear(A) \wedge Clear(B) \wedge \\
&\quad A \neq x \wedge x \neq B, s_0) \mid x \text{ is a constant}\} = \\
&1 + \min\{h_{max}(On(A,x) \wedge Clear(A) \wedge Clear(B), s_0) \mid x \text{ is a constant} \neq A, B\} = \\
&1 + \min\{h_{max}(On(A,C) \wedge Clear(A) \wedge Clear(B), s_0), \\
&\quad h_{max}(On(A,Table) \wedge Clear(A) \wedge Clear(B), s_0)\} = \\
&1 + h_{max}(On(A,Table) \wedge Clear(A) \wedge Clear(B), s_0) = \\
&1 + \max\{h_{max}(On(A,Table), s_0), h_{max}(Clear(A), s_0), h_{max}(Clear(B), s_0)\} = \\
&1 + \max\{0, h_{max}(Clear(A), s_0), 0\} = \\
&1 + h_{max}(Clear(A), s_0) = \\
&1 + (1 + h_{max}(On(C,A) \wedge Clear(C), s_0)) = \\
&1 + (1 + 0) = 2
\end{aligned}
$$

We hence get

$$
\begin{aligned}
h_{max}(s_0) &= h_{max}(g, s_0) = h_{max}(On(A,B) \wedge On(B,C), s_0) \\
&= \max\{h_{max}(On(A,B), s_0), h_{max}(On(B,C), s_0)\} \\
&= \max\{2, 1\} \\
&= 2.
\end{aligned}
$$

q) Is it possible to create a Blocks World problem where $h_{max}(s_0) > h^*(s_0)$?

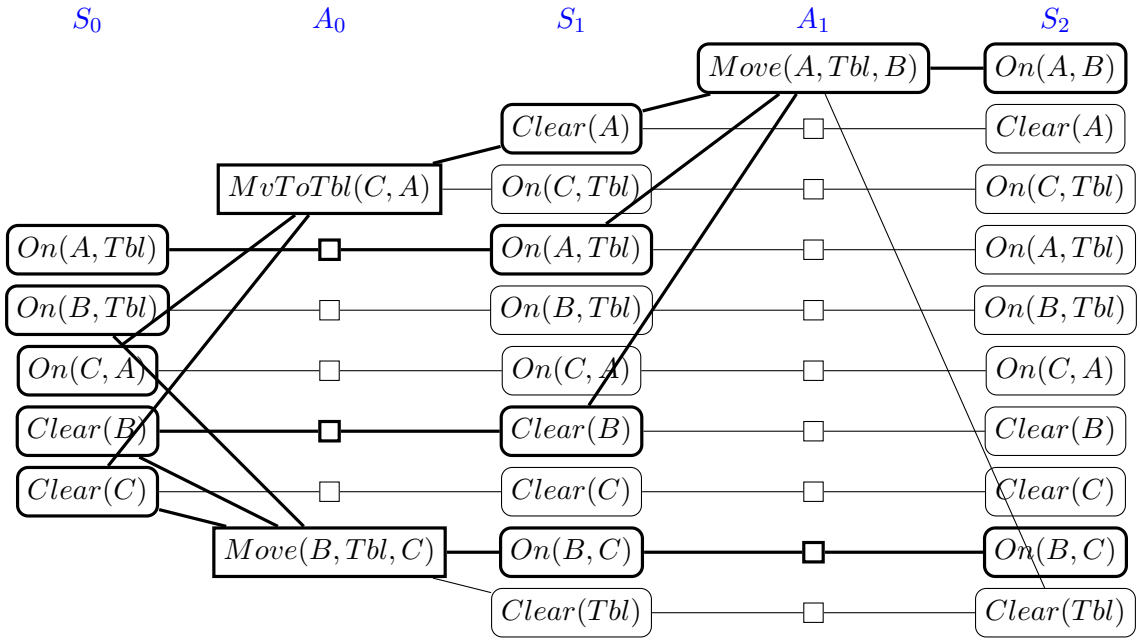
SOLUTION. No, the heuristic is admissible.

r) Find the admissible heuristics among the ones you have considered above (except h^*). Taking the maximum value of the value returned by all of these still gives an admissible heuristic (cf. the slides). What is this maximum value for the (rephrased) Blocks World problem considered in this exercise?

SOLUTION. It is 3, since $\max\{h_{ip}(s_0), h^+(s_0), h_{max}(s_0)\} = 3$. This is the optimal cost.

s) Compute $h_{FF}(s_0)$. When constructing the planning graph, you're allowed to omit states and actions that are not necessary to compute the value of $h_{FF}(s_0)$. Is any relaxed solution extracted from the planning graph a real solution to the problem?

SOLUTION.



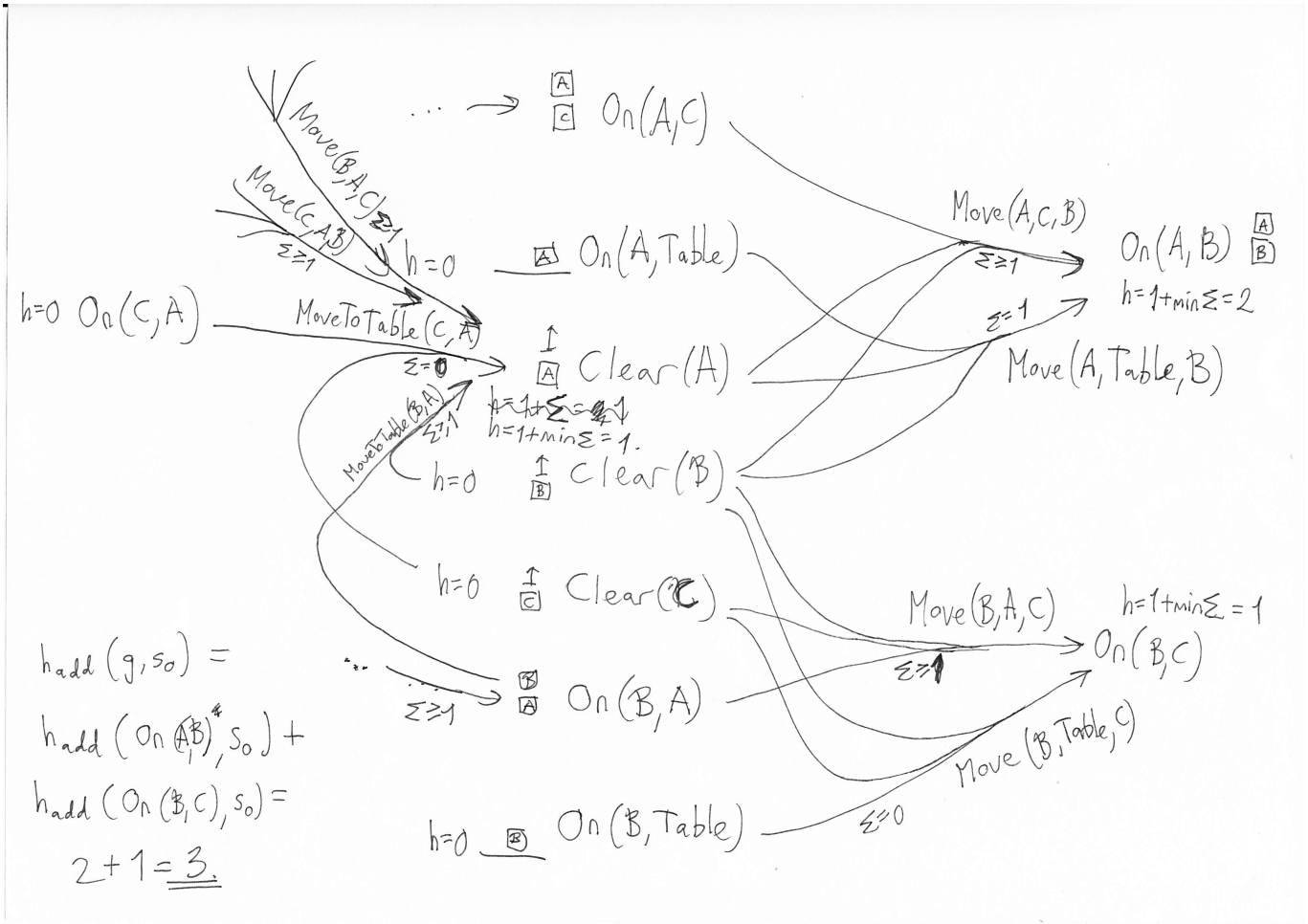
There are three chosen best supporting actions, so $h_{FF}(s_0) = 3$. The planning graph finds the correct three actions of the optimal solution to the problem, however it allows any order of the first two actions, only one of which is a solution. More precisely, from the planning graph, we might extract either of the following two relaxed plans:

$$MvToTbl(C, A), Move(B, Tbl, C), Move(A, Tbl, B)$$
$$Move(B, Tbl, C), MvToTbl(C, A), Move(A, Tbl, B)$$

where we note that only the first is a solution to the real problem.

t) *Optional*: Draw the (relevant part of the) directed hypergraph for calculating $h_{add}(s_0)$, cf. the slides from today and Section 2.7 of Geffner & Bonet. Look up directed hypergraph on Google first, if you don't know what they or how to draw them. To make the graph smaller, you can omit all nodes containing rigid predicates, as well as omit any action a in which one of the preconditions is false in s_0 and uses a rigid predicate.

SOLUTION.


On(A,C)
On(A,Table)
On(C,A)
On(B,A)
On(B,Table)
Clear(A)
Clear(B)
Clear(C)
MoveToTable(C,A)
MoveToTable(B,A)
Move(B,A,C)
Move(C,A,B)
Move(A,C,B)
Move(A,Table,B)
Move(B,A,C)
Move(B,Table,C)
On(A,B)
On(B,C)
h=0
Σ=0
Σ≥1
Σ=1
h=1+minΣ=2
h=1+minΣ=1
h_add(g,s0) =
h_add(On(A,B),s0) +
h_add(On(B,C),s0) =
2+1=3.